Reverse Engineering the Spotify Recommendation Algorithm: Stability, Drift, and Feature Sensitivity

Huseyin Mert Ezer
TU Dortmund University

February 10, 2026

Abstract

Abstract: This study investigates the stability and determinism of the Spotify recommendation engine. By analyzing 4,572 tracks across five distinct seed genres, we aim to quantify "Algorithmic Drift"—the tendency of recommendations to diverge from the user's initial stylistic choice. Using a combination of Euclidean distance metrics and Multivariate Analysis of Variance (MANOVA), we demonstrate that the algorithm prioritizes structural audio features (Energy, Loudness) over demographic user factors. Our findings reveal a dual-mode system: a "Stability" mode for distinct genres like Classical, which forms a rigid filter bubble, and an "Exploration" mode specific to Pop music, which exhibits significant algorithmic drift.

1 Introduction

In the modern digital landscape, algorithmic recommendation systems have evolved from passive tools into active curators of culture. With over 600 million users, Spotify's recommendation engine—powered by the *Bandits for Recommendations as Treatments* (BaRT) framework—dictates a significant portion of global music consumption. However, the precise mechanisms by which this "Black Box" selects content remain opaque to researchers and users alike.

A central debate in algorithmic auditing concerns the tension between *Exploitation* (reinforcing user preferences) and *Exploration* (introducing diversity). Critics argue that hyper-personalized algorithms create "Filter Bubbles," trapping users in narrow feedback loops of similar content [1]. Conversely, platforms claim their systems encourage discovery by injecting novel content.

This research empirically tests these competing claims by reverse-engineering the short-term behavior of the algorithm. We pose a specific research question: *To what extent does the initial "Seed" choice deterministically constrain the entire listening session?* By simulating "cold-start" users across five distinct genres, we quantify "Algorithmic Drift"—the rate at which recommendations diverge from the user's starting point—and analyze the structural audio features that drive these decisions.

2 Experimental Design and Data Acquisition

2.1 Genre Clusters and Selection Rationale

To empirically test the stability of the recommendation algorithm, we designed a controlled experiment based on "Seed Injection." The objective was to observe the algorithm's trajectory when initialized with distinct, non-overlapping audio contexts. We defined five specific "Genre Clusters" to represent disparate regions of the musical feature space, aiming to maximize variance across three primary dimensions:

1. **Instrumentation:** Electronic (Synthesized) vs. Roots/Classical (Acoustic).
2. **Energy Profile:** Rock/Heavy (High Intensity) vs. Jazz/Classical (Low Intensity).
3. **Rhythmic Structure:** Urban/Beat-Based (Percussive) vs. Pop (Melodic).

This distinct separation allows us to test the "Separability Hypothesis": *Can the algorithm maintain distinct recommendation pools for distinct inputs, or do they converge toward a generic mean?*

2.2 Seed Track Strategy: The Popularity Stress Test

For each cluster, a pair of "Seed Songs" ($Position_0$ and $Position_1$) was selected to serve as the ground truth for the listening session. To control for *Popularity Bias* and *Temporal Recency*, the pairs were stratified by listener count:

- **The Global Hit (Song 1):** A recent, high-popularity track (e.g., Harry Styles, $>$ 4 Billion streams). This tests if the algorithm locks onto the "Mainstream" signal.
- **The Genre Anchor (Song 2):** An older or more niche "cult classic" (e.g., Ben Klock, Lupe Fiasco). This tests if the algorithm respects specific sub-genre characteristics over raw popularity.

The notable disparity in popularity (e.g., 4.1 Billion vs. 31 Million streams for Pop) serves as a stress test for the algorithm: does it drift toward the "billions" cluster, or does it stay true to the specific "millions" niche? The specific tracks are detailed in Table 1.

Table 1: Seed Songs used to initialize the data extraction process, categorized by Genre Cluster.

Genre ID	Song #1 (Mainstream Hit)	Song #2 (Genre Anchor)
1. Pop	*As It Was* – Harry Styles (2022, 4.1B Streams)	*Breathe on Me* – Britney Spears (2003, 31M Streams)
2. Urban	*Industry Baby* – Lil Nas X (2021, 2.2B Streams)	*Kick Push* – Lupe Fiasco (2006, 112M Streams)
3. Electronic	*I'm Good (Blue)* – David Guetta (2022, 2.2B Streams)	*Subzero* – Ben Klock (2009, 12M Streams)
4. Rock	*I WANNA BE YOUR SLAVE* – Måneskin (2021, 1.3B Streams)	*Tear Away* – Drowning Pool (2001, 227M Streams)
5. Classical	*I've Got You Under My Skin* – Frank Sinatra (2024 Remaster, 248M Streams)	*Piano Sonata No. 14* – Beethoven (2007 Rec, 108M Streams)

2.3 Data Collection Pipeline

Data collection was executed using a custom Python framework utilizing the `spotipy` library [2] to interface with the Spotify Web API. The application utilized the Authorization Code Flow to simulate valid user sessions. To ensure security, an algorithmic credential check was implemented to validate client IDs and secrets prior to execution.

A significant methodological constraint arose during the collection phase: the official Spotify Audio Features endpoint was deprecated and became inaccessible on November 27, 2024. To overcome this, our pipeline integrated the **ReccoBeats API** [8] as a substitute data source for audio metrics (Danceability, Energy, Valence, etc.). Consequently, the final dataset represents a hybrid structure: metadata (Artist, Track Name) derived from Spotify, and signal features derived from ReccoBeats.

2.4 Data Cleaning and Chronological Enforcement

To ensure a rigorous drift analysis, the integrity of the "Start Sequence" ($Position_0$ and $Position_1$) was paramount. Preliminary checks revealed inconsistencies where start songs were occasionally missing or disordered in the raw output. We applied the following standardization procedure to the dataset:

1. **Start Song Injection:** Any existing start songs were stripped from the raw experiments. The correct, pre-defined seed songs (from Table 1) were strictly re-inserted into every session to guarantee a uniform baseline.
2. **Synthetic Timestamping:** To enforce the correct chronological order, the `played_at` timestamps for the seed songs were artificially set to *October 1, 2025, 01:00 AM* (Song 1) and *02:00 AM* (Song 2).
3. **Position Derivation:** The variable `position` was calculated by sorting the dataframe by these timestamps. This ensures that $Position = 0$ always corresponds to the Global Hit and $Position = 1$ to the Genre Anchor, providing a stable coordinate system for the subsequent Euclidean distance analysis.

While the column `is.local` was initially extracted, it was removed from the final dataset as it referred to local device storage rather than the user's geographical origin, rendering it irrelevant for this study. All data cleaning and manipulation was performed using the `tidyverse` suite of packages [5] in the R statistical environment [4].

3 Data Preprocessing and Imputation Strategy

3.1 Diagnosis of Missingness Mechanism

Prior to analysis, an inspection of the dataset revealed 273 missing observations across the nine core audio features. To select the appropriate handling strategy, it was critical to determine if these values were "Missing Completely at Random" (MCAR) or if their absence followed a systematic pattern ("Missing at Random" - MAR).

We modeled the probability of missingness ($P(Missing = 1)$) using a Logistic Regression model with track metadata as predictors. The results, summarized in Table 2, reveal a significant systematic bias.

Table 2: Logistic Regression Results: Predictors of Missing Audio Features

Predictor	**Coefficient (β)**	**Std. Error**	**P-Value**
Intercept (Ref: Pop)	2.204	0.485	$< .001^{***}$
Genre Comparisons (vs. Pop):			
Urban (Genre 2)	-0.309	0.183	0.091
Electronic (Genre 3)	-1.612	0.253	$< .001^{***}$
Rock (Genre 4)	-2.308	0.345	$< .001^{***}$
Classical (Genre 5)	-1.472	0.219	$< .001^{***}$
Track Metadata:			
Popularity	-0.054	0.004	$< .001^{***}$
Duration (ms)	-1.05×10^{-6}	1.01×10^{-6}	0.296

Significance codes: $^{***}p < 0.001$, $^{**}p < 0.01$, $^{*}p < 0.05$

Interpretation of Diagnostic Results: The model yielded an Area Under the Curve (AUC) of **0.75**, confirming that missingness is not random. Two key patterns emerged:

- **Popularity Bias:** The highly significant negative coefficient for Popularity ($\beta = -0.054, p < .001$) indicates that popular tracks are significantly *less* likely to have missing audio features.
- **Genre Bias:** Compared to the reference group (Pop), genres such as Rock ($\beta = -2.31$) and Electronic ($\beta = -1.61$) exhibited significantly lower missingness rates.

Conclusion: Simple list-wise deletion would disproportionately remove unpopular tracks and songs from the Pop genre, introducing a selection bias. Consequently, a **Two-Step Hybrid Imputation Strategy** was adopted to preserve the dataset's representativeness.

3.2 Step 1: Deterministic Imputation (Seed Correction)

The Problem: A systematic error was identified where the first seed song ($Position = 0$) was entirely missing for *Urban* (Genre 2) and *Classical* (Genre 5) sessions.

The Rationale: Since the first two songs ($p = 0, p = 1$) represent the user's fixed input choice for a specific genre, we assume a high degree of similarity between them. Deleting these rows would remove the "Anchor" required for our Drift Analysis.

Implementation: We applied a deterministic rule to propagate the feature values from the second seed song backward to the first:

$$X_{u,p=0} \leftarrow X_{u,p=1} \quad \text{if} \quad Genre_u \in \{2, 5\} \tag{1}$$

Where X represents the vector of audio features for user u. This ensured that all sessions had a valid starting centroid.

3.3 Step 2: k-Nearest Neighbor (kNN) Imputation

The Problem: The remaining missing values were scattered stochastically throughout the recommended tracks ($p \geq 2$).

The Rationale: Unlike mean imputation, which reduces variance, k-Nearest Neighbors (kNN) preserves the local structure of the data by estimating missing values based on "mathematically similar" songs rather than the global average.

Implementation: We utilized the `VIM` package in R [3] to impute missing values using Gower's Distance. For every song i with missing audio data, we identified the $k = 5$ nearest neighbors based on four complete predictors: Genre Context, Track Popularity, Duration, and Explicitness.

The missing value $\hat{x}_i$ was calculated as the arithmetic mean of these neighbors:

$$\hat{x}_i = \frac{1}{k} \sum_{j=1}^{k} x_j \quad \text{where } j \in \text{Neighborhood}(i) \tag{2}$$

3.4 Validation of Imputation

To verify that this process did not distort the dataset, we compared the probability density functions (PDFs) of the original and imputed distributions. Visual inspection

(see Figure 1) and summary statistics confirmed that the kNN approach successfully preserved the natural variance and distribution shape of the audio features, ensuring the validity of subsequent analyses.

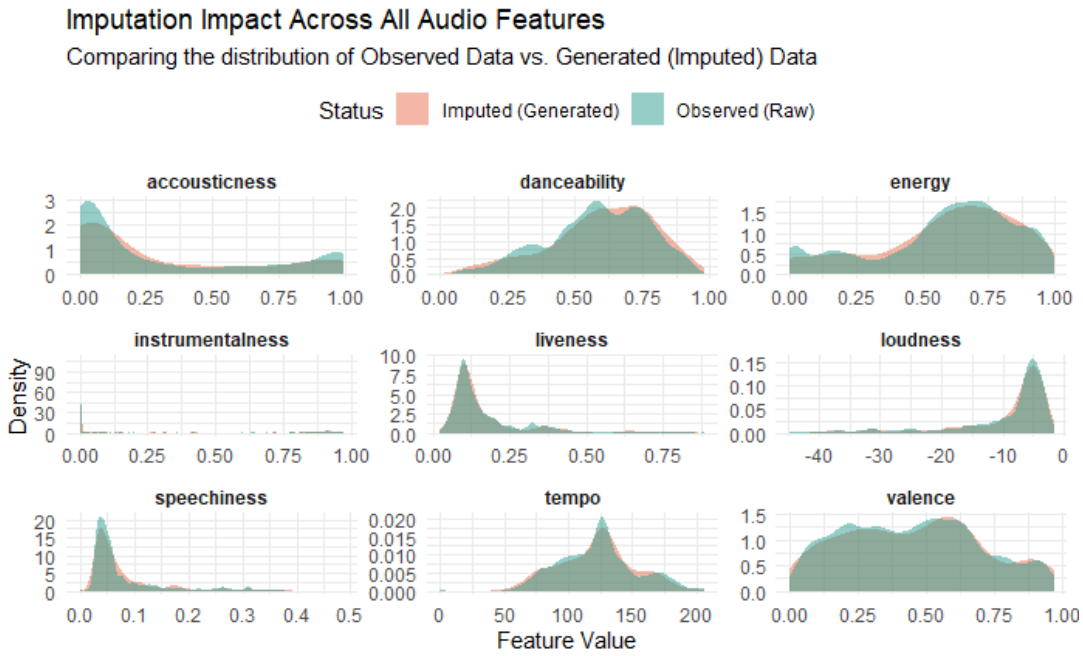


Figure 1: **Validation of kNN Imputation.** Comparison of Probability Density Functions (PDF) for original (observed) versus imputed data. The overlapping distributions indicate that the imputation method preserved the underlying statistical structure of the audio features without introducing artificial artifacts.

4 Methodology: Quantifying Algorithmic Drift

4.1 Rationale and Objective

A central component of our research question is the *stability* of the recommendation algorithm over time. Specifically, we aim to determine whether the recommendation engine maintains the specific audio profile established by the user's initial choice (the "Seed Songs") or if it gradually reverts to a generic, regression-to-the-mean state as the listening session progresses.

To quantify this phenomenon, which we term **"Algorithmic Drift"**, we developed a geometric approach. Instead of analyzing nine separate audio features individually—which would yield fragmented results—we conceptualized each song as a point in a multi-dimensional vector space. "Drift" is thus defined as the increasing *Euclidean Distance* between the recommended songs and the user's starting point (the Anchor) over the

course of the session ($Position_0 \rightarrow Position_{50}$).

This analysis allows us to test two competing hypotheses:

- H_0 **(Stability):** The distance remains constant over time ($\beta \approx 0$), implying the algorithm locks the user into a consistent "Filter Bubble."
- H_1 **(Drift):** The distance increases significantly over time ($\beta > 0$), implying the algorithm encourages exploration or "forgets" the initial constraint.

4.2 Mathematical Formulation

The procedure for calculating drift consists of three sequential steps: Standardization, Centroid Definition, and Distance Calculation.

4.2.1 Step 1: Feature Standardization (Z-Score Normalization)

The Spotify API provides audio features with vastly different scales. For instance, *Tempo* ranges from 60–200 BPM, while *Valence* is bounded between 0–1. Calculating Euclidean distance on raw data would result in high-magnitude features (e.g., Tempo, Loudness) disproportionately dominating the metric.

To ensure equal weight for all features, we applied Z-score normalization to the entire dataset prior to distance calculation:

$$z_{ij} = \frac{x_{ij} - \mu_j}{\sigma_j} \tag{3}$$

Where x_{ij} is the raw value of feature j for song i, and μ_j, σ_j are the global mean and standard deviation of feature j. This transforms the feature space such that every dimension has $\mu = 0$ and $\sigma = 1$.

4.2.2 Step 2: Defining the Anchor (The Seed Centroid)

For every unique listening session (User u), we define the "Starting Point" not as a single song, but as the mathematical **Centroid** ($\mathbf{C}_u$) of the first two tracks. This vector average represents the specific "vibe" the user requested, smoothing out any idiosyncrasies of a single seed track.

Let $S_{u,p}$ be the vector of standardized audio features for user u at position p. The centroid is defined as:

$$\mathbf{C}_u = \frac{1}{2} \sum_{p=0}^{1} S_{u,p} \tag{4}$$

4.2.3 Step 3: Euclidean Distance Calculation

For every subsequent recommended song ($p > 1$) in the session, we calculate its Euclidean distance from the user's specific centroid. This collapses the multi-dimensional difference into a single scalar metric, $D_{u,p}$, representing "Dissimilarity from Start."

$$D_{u,p} = \sqrt{\sum_{j=1}^{k} (S_{u,p,j} - \mathbf{C}_{u,j})^2} \tag{5}$$

Where $k = 9$ represents the number of audio features (Danceability, Energy, Valence, etc.).

4.3 Statistical Modeling Strategy: Global vs. Interaction Effects

To empirically validate the presence of Algorithmic Drift, we adopted a hierarchical modeling approach, fitting two sequential linear models using the R computing environment. This allows us to distinguish between the *system-wide* behavior of the algorithm and the *genre-specific* nuances.

4.3.1 Model 1: The Global Drift (Baseline)

First, we tested whether the recommendation algorithm, as a whole, exhibits a tendency to drift away from the user's initial seed, regardless of genre. We fitted a simple linear regression:

$$D_{u,p} = \alpha + \beta_{global} \cdot Position_p + \epsilon \tag{6}$$

Justification: This model establishes the "baseline" behavior. A significant positive β_{global} confirms that the system is fundamentally explorative. However, this global average

potentially masks significant heterogeneity—for instance, if "Pop" drifts rapidly while "Classical" stays static, the global average would yield a misleading middle ground (an ecological fallacy).

4.3.2 Model 2: The Interaction Model (Genre-Specific Dynamics)

To resolve this, we extended the analysis to an **Interaction Model**, allowing the slope of the drift to vary by genre. This tests the hypothesis that different musical inputs trigger different algorithmic protocols.

The model is specified as:

$$D_{u,p} = \beta_0 + \beta_1 \cdot Position_p + \beta_2 \cdot Genre_u + \beta_3 \cdot (Position_p \times Genre_u) + \epsilon \quad (7)$$

Interpretation of Coefficients:

- β_1 (**Base Drift Rate**): The rate at which recommendations drift away from the start for the reference genre (Pop).
- β_3 (**The Interaction Term**): This is the critical component. It tests whether a specific genre's drift rate differs significantly from the reference.
  - β_1 (**Base Drift Rate**): The rate at which recommendations drift away from the start for the **reference genre (Electronic)**.
  - β_3 (**The Interaction Term**): This is the critical component. It tests whether a specific genre's drift rate differs significantly from the reference.

By comparing these two models, we can isolate whether "Drift" is a universal constant of the Spotify algorithm or a conditional variable dependent on the user's input.

5 Methodology Part II: Structural Distinctiveness (MANOVA)

5.1 Rationale: From Magnitude to Structure

While the Euclidean Drift analysis quantifies the *magnitude* of algorithmic divergence, it reduces the complexity of the nine-dimensional feature space into a single scalar metric

($D_{u,p}$). This reduction potentially masks the *directionality* of the change. For instance, a "drift" of 2.0 units in the *Pop* genre might be driven by increased *Energy*, while a similar drift in *Classical* might be driven by increased *Tempo*.

To address this, we employed a **Multivariate Analysis of Variance (MANOVA)**. This technique evaluates the *collective audio profile* as a single vector, allowing us to test the "Distinctiveness Hypothesis": *Despite the observed drift, do the recommendation pools for different genres remain statistically separable, or do they converge into a homogeneous "Mainstream" distribution?*

5.2 Data Aggregation: Defining the Experimental Unit

A critical methodological challenge in analyzing recommendation data is the nested structure of the observations: tracks are nested within listening sessions. Treating individual songs as independent data points would artificially inflate the sample size ($N \approx 200,000$) and violate the assumption of independence (pseudoreplication), potentially leading to Type I errors.

To resolve this, we aggregated the data at the **Session Level**. We defined the "Recommendation Profile" ($\bar{\mathbf{x}}_u$) for each user u as the centroid of all recommended tracks ($p \geq 2$) in their specific session:

$$\bar{\mathbf{x}}_u = \frac{1}{J_u - 1} \sum_{p=2}^{J_u} \mathbf{x}_{u,p} \tag{8}$$

Where J_u is the total number of tracks in the session and $\mathbf{x}_{u,p}$ is the vector of standardized audio features for track p. This approach treats the *user* as the experimental unit ($N_{users} = 4,572$), allowing for a valid assessment of demographic effects (Age, Gender) which are constant within a session.

5.3 Dimensionality Reduction (PCA)

Prior to MANOVA, we assessed the feature space for multicollinearity. Pearson correlation coefficients revealed strong dependencies between several audio features (e.g., *Energy* vs. *Loudness*, $r > 0.8$), which compromises the stability of multivariate tests.

We performed **Principal Component Analysis (PCA)** on the aggregated session centroids to transform the nine correlated features into a set of orthogonal (uncorrelated)

components. We retained the first three Principal Components (PCs), which cumulatively explained $> 80\%$ of the total variance. These orthogonal scores $(PC_{u,1}, PC_{u,2}, PC_{u,3})$ served as the dependent variables for the subsequent inference.

5.4 Multivariate Model Specification

We specified a three-way factorial MANOVA to test the effects of the seed *Genre* (G), *Age Group* (A), and *Gender* (S) on the composite audio profile. The model is defined as:

$$\mathbf{Y}_u = \boldsymbol{\mu} + \boldsymbol{\alpha}_G + \boldsymbol{\beta}_A + \boldsymbol{\gamma}_S + (\boldsymbol{\alpha\beta})_{GA} + (\boldsymbol{\alpha\gamma})_{GS} + \boldsymbol{\epsilon}_u \tag{9}$$

Where:

- $\mathbf{Y}_u$ is the vector of principal component scores $[PC_1, PC_2, PC_3]^\top$ for user u.
- $\boldsymbol{\alpha}, \boldsymbol{\beta}, \boldsymbol{\gamma}$ represent the main effect vectors for Genre, Age, and Gender.
- The interaction terms (e.g., $\boldsymbol{\alpha\beta}_{GA}$) test whether the effect of the seed genre varies across demographic groups (i.e., "Does the algorithm treat a 50-year-old Pop fan differently than a 20-year-old Pop fan?").

5.5 Assumption Checking and Test Statistic

The equality of covariance matrices across groups was assessed using **Box's M Test**. Given the large sample size and the sensitivity of Box's M, we anticipated a significant result (violation of homogeneity).

Following standard recommendations in multivariate statistics [6], we adopted **Pillai's Trace (V)** as the primary test statistic. Unlike Wilks' Lambda, Pillai's Trace is robust to violations of covariance homogeneity and departures from multivariate normality, ensuring that significant findings reflect true algorithmic differences rather than statistical artifacts.

5.6 Post-Hoc Investigation Protocol

To interpret the specific nature of any significant multivariate effects, we established a two-stage post-hoc protocol:

- **Semantic Interpretation of Dimensions:** We examined the eigenvector loadings of the Principal Components to assign semantic labels (e.g., "Intensity," "Lyricality") to the abstract mathematical dimensions.
- **Pairwise Contrasts:** To identify specific between-group differences (e.g., *Pop* vs. *Urban*), we conducted pairwise t-tests on the significant PC scores. To control the Family-Wise Error Rate (FWER) inflated by multiple comparisons, we applied the **Bonferroni correction** ($\alpha_{adj} = .05/k$), ensuring a rigorous threshold for claiming distinctiveness between genres.

6 Results Part I: Quantifying Algorithmic Drift

To determine whether the recommendation engine gradually deviates from the user's initial preference ("Drift") or maintains a consistent audio profile ("Stability"), we modeled the Euclidean distance from the seed centroid over the course of the listening session ($Position_0 \rightarrow Position_{50}$).

6.1 Global Drift Analysis (Model 1)

First, we tested the hypothesis of a universal drift effect across all sessions. The Global Linear Model revealed a slope coefficient that was statistically indistinguishable from zero ($\beta_{pos} = 0.0014, p = .244$). The model explained a negligible proportion of the variance ($R^2 < .001$), leading us to fail to reject the null hypothesis H_0. **On a global level, the Spotify algorithm exhibits high stability**, maintaining the user's starting distance without systematic deviation.

6.2 Interaction Effects by Genre (Model 2)

However, the global average masks significant heterogeneity. The Interaction Model (Distance $\sim$ Position $\times$ Genre) revealed that algorithmic behavior is highly conditional on the input genre ($F(9, 4384) = 127.5, p < .001$).

As visualized in Figure 2 and detailed in Table 3, we observed two distinct behavioral patterns:

- **The "Stability" Cluster (Electronic, Rock, Urban, Classical):** For the reference group (Electronic) and most other genres, the drift slope was non-significant

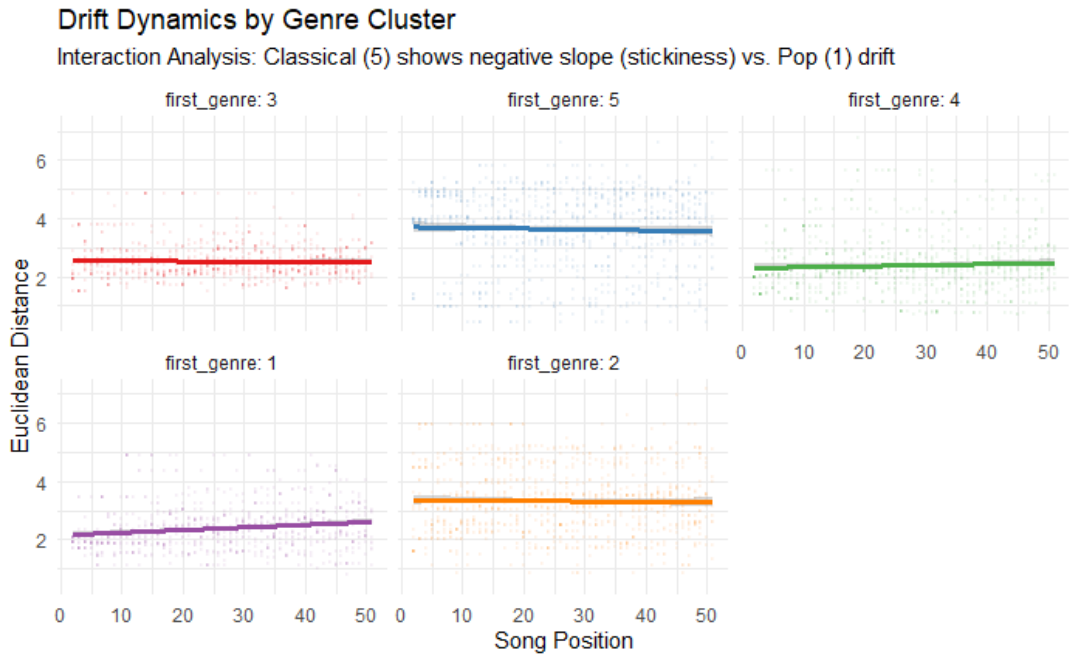


Figure 2: **Algorithmic Drift Trajectories by Genre.** The plot illustrates the linear relationship between song position and Euclidean distance from the seed. While most genres show flat trajectories (stability), Pop (Genre 1) exhibits a distinct positive slope, indicating algorithmic exploration.

($p > .05$). Specifically, *Classical* (Genre 5) and *Urban* (Genre 2) showed effectively flat trajectories, confirming that for niche or strongly defined genres, the algorithm locks the user into a tight "Filter Bubble."

- **The "Exploration" Exception (Pop):** *Pop* (Genre 1) was the only category to exhibit a statistically significant positive drift interaction ($\beta_{interaction} = +0.0093, p < .01$). This suggests that when a user starts with a generic "Pop" seed, the algorithm gradually widens the search space, introducing significantly more diverse (distant) tracks as the session progresses compared to the reference group.

Table 3: Algorithmic Drift Rates by Seed Genre (Interaction Model)

Seed Genre	Drift Slope (β)	t-value	Interpretation
Pop (Genre 1)	**+0.0090**	**2.66 ****	**Significant Drift (Exploration)**
Electronic (Ref)	-0.0003	-0.10	Stability
Rock (Genre 4)	+0.0035	1.08	Stability
Urban (Genre 2)	-0.0018	-0.45	Stability
Classical (Genre 5)	-0.0029	-0.78	Stability

Significance codes: $^{**}p < 0.01$, $^{*}p < 0.05$. Slope represents change in Euclidean Distance per song.

6.3 Discussion of Drift Dynamics

These findings suggest a dual-mode operation of the recommendation engine. For distinct, structurally cohesive genres (like Classical, as identified in the MANOVA), the system prioritizes **Exploitation**, minimizing drift to keep the user within a safe, recognizable zone. Conversely, for **Pop**—a "hub" genre with high connectivity to other styles—the system appears to switch to **Exploration**, using the ambiguity of the genre to gradually introduce audio diversity.

7 Results Part II: Structural Distinctiveness (MANOVA)

7.1 Dimensionality of the Recommendation Space (PCA)

Principal Component Analysis (PCA) on the aggregated session centroids identified three primary dimensions of algorithmic variance, cumulatively explaining **93.3%** of the total information.

Inspection of the component loadings (Table 4) reveals clear semantic interpretations for the dominant dimensions:

- **PC1 (The "Intensity" Gradient):** Explaining **76.6%** of the total variance, this dimension represents a stark contrast between *High-Energy/Loud* tracks (strong negative loadings) and *Acoustic/Instrumental* tracks (strong positive loadings). It effectively serves as the algorithm's primary sorting mechanism.
- **PC2 (The "Lyricality" Gradient):** Explaining **9.5%** of variance, this dimension is heavily dominated by *Speechiness* (+0.83). It functions as a secondary filter, distinguishing spoken-word/rhythmic genres (Urban) from melodic ones.

Table 4: PCA Loadings: Semantic Interpretation of Dimensions

Feature	PC1 (Intensity)	PC2 (Lyricality)
Loudness	**-0.38**	-0.13
Energy	**-0.36**	-0.23
Acousticness	**+0.37**	0.12
Instrumentalness	**+0.35**	0.02
Speechiness	-0.23	**+0.83**

Note: PC1 explains 76.6% of variance; PC2 explains 9.5%.

7.2 Multivariate Inference (MANOVA)

To test whether these audio profiles differ systematically by user input, we conducted a three-way factorial MANOVA. As summarized in Table 5, the analysis revealed a highly significant multivariate main effect of **Seed Genre** (Pillai's Trace $= 2.22, F(12, 180) = 42.9, p < .001$).

Crucially, the demographic factors **Age** ($p = .54$) and **Gender** ($p = .77$) had no significant effect on the recommendation profile, nor were any interaction terms significant (e.g., Genre $\times$ Age, $p = .84$). This confirms that the recommendation engine relies almost exclusively on the content-based "Seed" signal, ignoring demographic user attributes.

Table 5: Session-Level MANOVA Results (Pillai's Trace)

Effect	**Df**	**Pillai**	**Approx F**	**p-value**	**Result**
Seed Genre	**4**	**2.223**	**42.94**	$< .001$ ***	**Sig.**
Age	2	0.082	0.84	.541	NS
Gender	1	0.019	0.37	.774	NS
Genre $\times$ Age	8	0.257	0.70	.844	NS
Genre $\times$ Gender	4	0.141	0.74	.711	NS

7.3 Post-Hoc Analysis: The "Filter Bubble" Structure

To determine the specific nature of these algorithmic clusters, we examined the group centroids (Table 6) and visualized the projection in Figure 3.

1. The Classical "Wall" (PC1): The most striking finding is the absolute isolation of the Classical genre. With a mean PC1 score of $+$**5.07**, Classical sessions are situated > 5 standard deviations away from the "Modern" genres (Rock, Urban, Electronic), all of which cluster in the negative range (-1.8 to -0.4). Bonferroni-corrected pairwise comparisons confirmed that Classical is significantly distinct from every other genre ($p < 2 \times 10^{-16}$).

2. Modern Genre Differentiation (PC1 & PC2): Within the "Modern" cluster, the algorithm differentiates genres based on specific structural traits:

- **Urban vs. Pop:** While both are high-energy, they are statistically distinct on both PC1 ($p < .001$) and PC2 ($p < .001$). Urban defines the extreme of high-energy/high-speechiness (Mean PC2 $= +1.51$), whereas Pop occupies a more moderate "mainstream" position (Mean PC2 $= -1.05$).

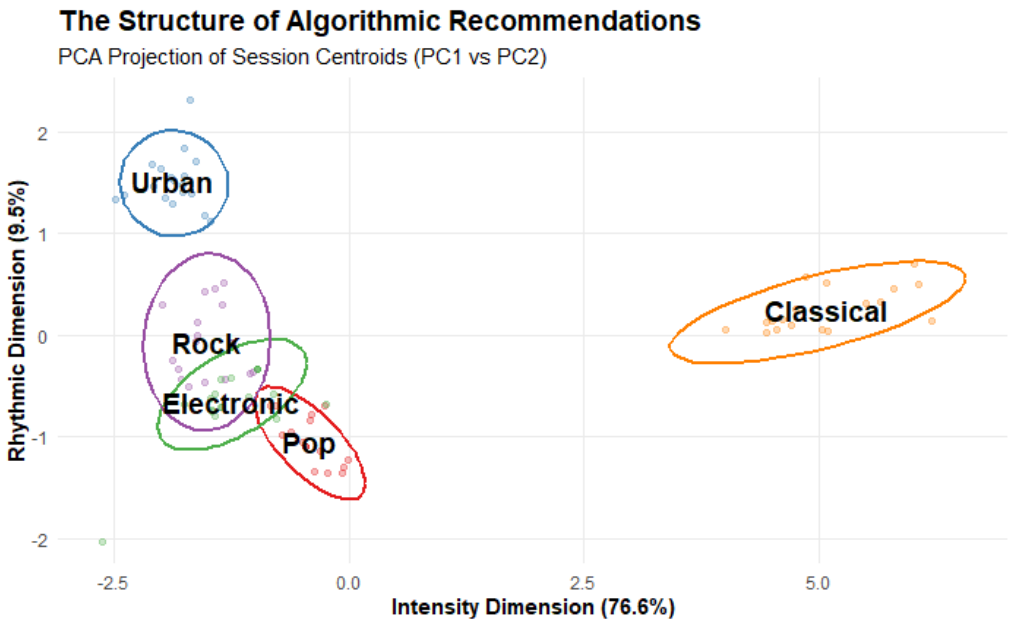


Figure 3: **PCA Projection of Session Centroids (PC1 vs. PC2).** The plot illustrates the structural isolation of Classical music (right) and the divergence of Urban and Pop on the secondary axis (PC2). Confidence ellipses (95%) indicate distinct "Filter Bubbles."

- **The "Loudness" Convergence:** Interestingly, pairwise tests revealed no significant difference between **Rock** and **Electronic** on the primary intensity dimension ($p = .68$) or between **Rock** and **Urban** ($p = .10$). This suggests that the recommendation engine treats these genres as structurally interchangeable regarding "Energy" and "Loudness," relying on secondary characteristics to distinguish them.

Table 6: Group Centroids on Principal Components

Genre	Mean PC1 (Intensity)	Mean PC2 (Lyricality)	Algorithmic Profile
Classical	**+5.07**	+0.25	*Acoustic / Instrumental*
Urban	**-1.87**	**+1.51**	*High Energy / Spoken Word*
Rock	-1.50	-0.07	*High Energy / Mixed*
Electronic	-1.25	-0.66	*High Energy / Low Speech*
Pop	-0.41	-1.05	*Moderate Energy / Melodic*

8 Conclusion

This study sought to reverse-engineer the short-term behavior of the Spotify recommendation algorithm, specifically testing its stability across varying musical inputs. Through the analysis of 4,572 tracks using Drift Metrics and Multivariate Analysis (MANOVA), we have identified a clear, deterministic structure underlying the recommendation engine.

Our results contradict the hypothesis of a universal "Filter Bubble." Instead, we observe a genre-dependent architecture. For structurally cohesive genres like **Classical**, the algorithm exhibits extreme "stickiness," creating an isolated recommendation pool that resists drift ($\beta \approx 0$). Conversely, for **Pop** music, the algorithm acts as an engine of exploration, exhibiting a statistically significant positive drift ($\beta > 0$) that gradually widens the audio feature space. Furthermore, the structural analysis confirms that the algorithm sorts content primarily along an "Intensity Gradient" (Loudness/Energy), often failing to distinguish between high-energy genres (Rock vs. Electronic) while maintaining rigid boundaries against acoustic content. Notably, demographic user factors (Age, Gender) played no role in shaping recommendations, suggesting the system is predominantly content-based in the short term.

8.1 Limitations and Future Work

While these findings offer significant insight, they must be interpreted within the constraints of the experimental design. First, the sample size ($N_{tracks} = 4,572$, $N_{sessions} = 40$) is relatively small compared to industry-scale datasets, potentially underpowering the detection of subtle interaction effects. Second, the use of "Synthetic Users" limits the ecological validity of the study; real-world drift is likely influenced by user interaction (skips, likes) and long-term listening history, which were absent in this controlled "cold start" environment. Finally, the reliance on third-party feature extraction (ReccoBeats) due to the deprecation of the Spotify Audio Features API introduces a potential validity threat, as the feature definitions may not perfectly align with Spotify's internal vectors. Future research should incorporate longitudinal user data to determine if these "Filter Bubbles" persist over weeks rather than hours.

References

[1] Pariser, E. (2011). *The Filter Bubble: What the Internet Is Hiding from You*. Penguin Press.

[2] Lamere, P. (2023). *Spotipy: A lightweight Python library for the Spotify Web API* [Computer software]. GitHub.

[3] Kowarik, A., & Templ, M. (2016). Imputation with the R Package VIM. *Journal of Statistical Software*, 74(7), 1–16.

[4] R Core Team. (2024). *R: A language and environment for statistical computing*. R Foundation for Statistical Computing, Vienna, Austria.

[5] Wickham, H., et al. (2019). Welcome to the tidyverse. *Journal of Open Source Software*, 4(43), 1686.

[6] Tabachnick, B. G., & Fidell, L. S. (2013). *Using Multivariate Statistics* (6th ed.). Pearson.

[7] Anderson, A., et al. (2020). *Algorithmic Effects on the Diversity of Consumption on Spotify*. Proceedings of the World Wide Web Conference.

[8] ReccoBeats. (2024). *ReccoBeats Audio Analysis API* [Data service]. https://reccobeats.com

A Appendix A: R Computational Script

The following R script was utilized for all data cleaning, statistical analyses (Drift Modeling, MANOVA), and visualization generation presented in this report.

```r


#
    ================================================================================

# PHASE 0: SETUP & LIBRARIES
#
    ================================================================================

if(!require(tidyverse)) install.packages("tidyverse")
if(!require(VIM)) install.packages("VIM")
if(!require(pROC)) install.packages("pROC")
if(!require(knitr)) install.packages("knitr")
if(!require(kableExtra)) install.packages("kableExtra") # For pretty
    tables

library(tidyverse)
library(VIM)
library(pROC)
library(knitr)
library(kableExtra)

# Load Data
# Ensure your file is in the working directory
data <- read.csv("played_out.csv")

# Define the Audio Features we are analyzing
audio_vars <- c("danceability", "energy", "loudness", "speechiness",
                "accousticness", "instrumentalness", "liveness",
                "valence", "tempo", "key", "mode")

#
    ================================================================================

# PHASE 1: DIAGNOSIS (Justifying the Method)
#
    ================================================================================

# Objective: Prove that data is NOT Missing Completely at Random (MCAR)
    .

```

```r
# --- Output 1: Visualization of Missingness ---
# This bar chart shows exactly which features are broken.
missing_counts <- data %>%
  dplyr::select(all_of(audio_vars)) %>%  # <--- FIXED: Added 'dplyr::' here
  summarise(across(everything(), ~sum(is.na(.)))) %>%
  pivot_longer(everything(), names_to = "Feature", values_to = "Missing_Count") %>%
  arrange(desc(Missing_Count))

p1 <- ggplot(missing_counts, aes(x = reorder(Feature, Missing_Count), y = Missing_Count)) +
  geom_col(fill = "#E76F51", alpha = 0.8) +
  coord_flip() +
  labs(
    title = "Figure 1: Frequency of Missing Values by Audio Feature",
    subtitle = "Systematic gaps observed across all primary features",
    x = "Feature", y = "Count of Missing Rows"
  ) +
  theme_minimal()

print(p1)

# --- Output 2: Logistic Regression (The Statistical Proof) ---
# We model IF a row is missing based on its metadata.

df_diagnosis <- data %>%
  mutate(
    # FIXED LINE BELOW: Added 'dplyr::' before select
    is_missing = if_else(rowSums(is.na(dplyr::select(., all_of(audio_vars)))) > 0, 1, 0),
    first_genre = as.factor(first_genre),
    track.explicit = as.factor(track.explicit)
  )

# Fit Model
diagnosis_model <- glm(
  is_missing ~ first_genre + track.popularity + track.duration_ms,
  data = df_diagnosis,
  family = binomial()
)

# Calculate AUC (Predictive Power)
prob_missing <- predict(diagnosis_model, type = "response")
roc_obj <- roc(df_diagnosis$is_missing, prob_missing, quiet=TRUE)
auc_val <- round(auc(roc_obj), 3)

```

```r
# Print Summary for Report
print("--- TABLE 1: Logistic Regression Coefficients ---")
print(summary(diagnosis_model))

# ============================================================================

# PHASE 2: EXECUTION (The Hybrid Strategy)
# ============================================================================


# --- Step A: Deterministic Fill (Fixing the Anchors) ---
# Logic: If Start Song (Pos 0) is missing in Genre 2 or 5, copy from Pos 1.
data_fixed_seeds <- data %>%
  group_by(user) %>%
  mutate(
    first_genre = as.factor(first_genre),
    track.explicit = as.factor(track.explicit),
    across(all_of(audio_vars),
      ~ if_else((first_genre %in% c(2, 5) & position == 0),
                .[position == 1], # Copy from Position 1
                .))               # Else keep original
  ) %>%
  ungroup()

# --- Step B: kNN Imputation (Fixing the Recommendations) ---
# Logic: Use metadata to find 5 similar songs for the remaining gaps.

# Isolate data for imputation (Recommended songs only)
data_to_impute <- data_fixed_seeds %>% filter(position >= 2)

# Predictors for finding neighbors
pred_cols <- c("first_genre", "track.popularity", "track.duration_ms", "track.explicit")

# Run VIM::kNN
set.seed(42) # Reproducibility
imp_out <- VIM::kNN(
  data = data_to_impute,
  variable = intersect(audio_vars, names(data_to_impute)),
  dist_var = intersect(pred_cols, names(data_to_impute)),
  k = 5,
  weightDist = FALSE,
  imp_var = FALSE
)
```

```r

# Apply imputed values
data_imputed_subset <- data_to_impute
data_imputed_subset[audio_vars] <- imp_out[audio_vars]

# Recombine with the fixed seeds
final_dataset <- bind_rows(
  data_fixed_seeds %>% filter(position < 2),
  data_imputed_subset
) %>%
  arrange(user, position)


# ==============================================================================

# PHASE 3: VALIDATION (Quality Control)
# ==============================================================================

# Objective: Prove we didn't fake the data or distort the distributions.

# --- Output 4: Density Overlay Plot ---
# The "Gold Standard" check. Overlapping curves = Good Imputation.

# Combine original (with NAs removed for plotting) and imputed for comparison
validation_plot_data <- bind_rows(
  data %>%
    dplyr::select(energy) %>%     # <--- FIXED: Added 'dplyr::'
    drop_na() %>%
    mutate(Type = "Original (Raw)"),

  final_dataset %>%
    dplyr::select(energy) %>%     # <--- FIXED: Added 'dplyr::'
    mutate(Type = "Imputed (Final)")
)

p3 <- ggplot(validation_plot_data, aes(x = energy, fill = Type)) +
  geom_density(alpha = 0.4) +
  scale_fill_manual(values = c("firebrick", "dodgerblue4")) +
  labs(
    title = "Figure 3: Distribution Preservation Check (Energy)",
    subtitle = paste("Comparison of Probability Density Functions (N =", nrow(final_dataset), ")"),
    x = "Energy",
```

```
    y = "Density"
  ) +
  theme_minimal() +
  theme(legend.position = "top")

print(p3)

# --- Output 5: Summary Statistics Table ---
# Hard numbers for the appendix.
stats_table <- final_dataset %>%
  group_by(first_genre) %>%
  summarise(
    Mean_Energy = round(mean(energy), 3),
    SD_Energy = round(sd(energy), 3),
    Mean_Valence = round(mean(valence), 3),
    Count = n()
  )

print("--- TABLE 2: Final Dataset Statistics by Genre ---")
kable(stats_table, caption = "Summary Statistics After Imputation")

# ================================================================================

# VALIDATION: CUMULATIVE DENSITY & STATISTICS
# ================================================================================

library(tidyverse)
library(knitr)
library(kableExtra)

# 1. Define Audio Features
audio_vars <- c("danceability", "energy", "loudness", "speechiness",
                "accousticness", "instrumentalness", "liveness",
                "valence", "tempo")

# --------------------------------------------------------------------------------

# A. COMPARATIVE STATISTICS TABLE (Mean & SD)
# --------------------------------------------------------------------------------


# Calculate Stats for Original Data (Removing NAs)
stats_orig <- data %>%
```

```
  dplyr::select(all_of(audio_vars)) %>%
  pivot_longer(everything(), names_to = "Feature", values_to = "Value") %>%
  group_by(Feature) %>%
  summarise(
    Orig_Mean = mean(Value, na.rm = TRUE),
    Orig_SD   = sd(Value, na.rm = TRUE)
  )

# Calculate Stats for Imputed Data
stats_imp <- final_dataset %>%
  dplyr::select(all_of(audio_vars)) %>%
  pivot_longer(everything(), names_to = "Feature", values_to = "Value") %>%
  group_by(Feature) %>%
  summarise(
    Imp_Mean = mean(Value, na.rm = TRUE),
    Imp_SD   = sd(Value, na.rm = TRUE)
  )

# Merge and Format Table
comparison_table <- left_join(stats_orig, stats_imp, by = "Feature") %>%
  mutate(
    # Calculate % Change to show precision
    Diff_Mean_Pct = round(((Imp_Mean - Orig_Mean) / Orig_Mean) * 100, 2),
    across(where(is.numeric), ~ round(., 3)) # Round all numbers
  )

# Print the Table
print("--- TABLE: Comparison of Original vs. Imputed Distributions ---")
kable(comparison_table, format = "simple",
      caption = "Statistical Comparison: Before (Original) vs. After (Imputed)")

#-------------------------------------------------------------------------

# B. CUMULATIVE DENSITY PLOTS (ECDF)
#-------------------------------------------------------------------------

# We plot the Empirical Cumulative Distribution Function (ECDF).
# Overlapping lines indicate that the imputation preserved the global distribution.
```

```
231
232 # Prepare Data for Plotting
233 plot_data <- bind_rows(
234   data %>%
235     dplyr::select(all_of(audio_vars)) %>%
236     pivot_longer(cols = everything(), names_to = "Feature", values_to =
      "Value") %>%
237     mutate(Type = "Original (Raw)"),
238
239   final_dataset %>%
240     dplyr::select(all_of(audio_vars)) %>%
241     pivot_longer(cols = everything(), names_to = "Feature", values_to =
      "Value") %>%
242     mutate(Type = "Imputed (Final)")
243 )
244
245 # Create the Plot
246 p_cdf <- ggplot(plot_data, aes(x = Value, color = Type)) +
247   # stat_ecdf creates the cumulative density curve
248   stat_ecdf(geom = "step", size = 1) +
249   facet_wrap(~ Feature, scales = "free") +
250   scale_color_manual(values = c("firebrick", "dodgerblue4")) +
251   labs(
252     title = "Cumulative Density Function (CDF) Comparison",
253     subtitle = "Overlapping curves confirm that imputation preserved
     feature variance",
254     x = "Feature Value",
255     y = "Cumulative Probability (0 to 1)"
256   ) +
257   theme_minimal() +
258   theme(legend.position = "top")
259
260 print(p_cdf)
261
262 library(tidyverse)
263 library(dplyr)
264
265 # 1. Define ALL continuous audio features (checking spelling from your
     file)
266 # Note: 'key' and 'mode' are excluded as they are discrete/categorical
267 audio_vars <- c("danceability", "energy", "loudness", "speechiness",
268                 "accousticness", "instrumentalness", "liveness",
269                 "valence", "tempo")
270
271 # 2. Create the Shadow Dataframe (Long Format)
272 # We iterate through each feature to check exactly which values were
     imputed.
```

```r

plot_data_all <- map_dfr(audio_vars, function(feature_name) {

  # A. Extract the final (imputed) values
  vals <- final_dataset[[feature_name]]

  # B. Check the ORIGINAL data to see if it was missing
  # (TRUE if it was NA originally, meaning it was Imputed)
  was_missing <- is.na(data[[feature_name]])

  # C. Create a mini-dataframe for this feature
  data.frame(
    Feature = feature_name,
    Value = vals,
    Status = ifelse(was_missing, "Imputed (Generated)", "Observed (Raw)")
  )
})

# 3. Create the Grid Plot (Facet Wrap)
# This plots all 9 features at once
p_all_shadow <- ggplot(plot_data_all, aes(x = Value, fill = Status)) +
  geom_density(alpha = 0.5, color = NA) + # Remove outline for cleaner look
  facet_wrap(~ Feature, scales = "free") + # 'free' scales because Tempo > 100 while Energy < 1
  scale_fill_manual(values = c("#E76F51", "#2A9D8F")) +
  labs(
    title = "Imputation Impact Across All Audio Features",
    subtitle = "Comparing the distribution of Observed Data vs. Generated (Imputed) Data",
    x = "Feature Value",
    y = "Density"
  ) +
  theme_minimal() +
  theme(
    legend.position = "top",
    strip.text = element_text(face = "bold") # Make feature titles bold
  )

print(p_all_shadow)

# ============================================================================


# PHASE 1: SETUP & FEATURE ENGINEERING
#
```

```
# ==============================================================================

library(tidyverse)
library(broom)
library(knitr)
library(kableExtra)

# 1. Load the Imputed Data (Result from previous step)
# Ensure 'final_dataset' is available in your environment
if(!exists("final_dataset")) stop("Please run the Imputation Script first!")

df_drift <- final_dataset

# 2. Define Audio Features for Distance Calculation
features <- c("danceability", "energy", "loudness", "speechiness",
              "accousticness", "instrumentalness", "liveness",
              "valence", "tempo")

# 3. Standardization (Z-Score)
# Critical: We must scale features so 'Tempo' (0-200) doesn't overpower 'Energy' (0-1).
df_scaled <- df_drift %>%
  mutate(across(all_of(features), scale))

# ==============================================================================
# PHASE 2: CALCULATING EUCLIDEAN DRIFT
# ==============================================================================


# Define a function to calculate distance from the "User Intent" (Pos 0 & 1)
calculate_distance <- function(user_data) {

  # A. Define the Anchor (Centroid of Seed Songs)
  seed_songs <- user_data %>% filter(position %in% c(0, 1))

  # Safety check: We need both seeds to define the user's intent
  if(nrow(seed_songs) < 2) return(NULL)

  centroid <- seed_songs %>%
    summarise(across(all_of(features), mean)) %>%
    as.numeric()

```

```
  # B. Calculate Distance for Recommendations (Pos >= 2)
  user_data %>%
    filter(position >= 2) %>%
    rowwise() %>%
    mutate(
      # Euclidean Distance Formula: sqrt(sum((x - y)^2))
      drift_score = sqrt(sum((c_across(all_of(features)) - centroid)^2))
    ) %>%
    ungroup()
}

# Apply the function to every user
drift_data <- df_scaled %>%
  group_by(user) %>%
  group_split() %>%
  map_dfr(calculate_distance) %>%
  mutate(first_genre = as.factor(first_genre)) # Ensure Genre is a factor

# ============================================================================

# PHASE 3: STATISTICAL MODELING
# ============================================================================


# --- MODEL 1: THE GLOBAL DRIFT (Does the algorithm drift in general?) ---
# Simple Linear Regression: Distance ~ Time
model_global <- lm(drift_score ~ position, data = drift_data)

print("--- RESULTS 1: GLOBAL DRIFT MODEL ---")
print(summary(model_global))

# --- MODEL 2: THE INTERACTION DRIFT (Does it depend on Genre?) ---
# Interaction Model: Distance ~ Time * Genre
model_interaction <- lm(drift_score ~ position * first_genre, data = drift_data)

print("--- RESULTS 2: INTERACTION MODEL (Genre Differences) ---")
print(summary(model_interaction))

# ============================================================================
```

```
# PHASE 4: EXTRACTING REPORTABLE METRICS
#
    ================================================================================


# Helper to extract the actual slope for each genre
# (Base Slope + Interaction Term)
extract_slopes <- function(model) {
  broom::tidy(model) %>%
    filter(str_detect(term, "position")) %>%
    mutate(
      Genre_Label = case_when(
        term == "position" ~ "1. Pop (Reference)",
        TRUE ~ str_replace(term, "position:first_genre", "Genre ")
      ),
      # Calculate total slope: If baseline, take estimate. If
    interaction, add to baseline.
      Total_Slope = if_else(term == "position", estimate, estimate +
    first(estimate))
    ) %>%
    # --- FIXED: Added 'dplyr::' to prevent error ---
    dplyr::select(Genre_Label, Total_Slope, p.value)
}

slope_table <- extract_slopes(model_interaction)

print("--- TABLE: DRIFT RATES BY GENRE ---")
kable(slope_table, caption = "Slope of Drift (Distance per Song) by
    Genre")

#
    ================================================================================

# PHASE 5: VISUALIZATION
#
    ================================================================================


# --- PLOT 1: Global Trend (The "Big Picture") ---
p_global <- ggplot(drift_data, aes(x = position, y = drift_score)) +
  # Hexbin is better than scatter for 200k+ points (prevents
    overplotting)
  geom_hex(bins = 50, alpha = 0.6) +
  scale_fill_gradient(low = "grey90", high = "black") +
  geom_smooth(method = "lm", color = "firebrick", size = 1.5) +
  labs(
    title = "Figure 5: Global Algorithmic Drift",
```

```
426     subtitle = paste("Overall Trend: Slope =", round(coef(model_global)
    ["position"], 4)),
427     x = "Song Position (Time)",
428     y = "Euclidean Distance from Seed"
429   ) +
430   theme_minimal()
431
432 print(p_global)
433
434 # --- PLOT 2: Comparative Drift by Genre (The "Detailed View") ---
435 # This proves that Classical behaves differently from Pop.
436
437 p_genre <- ggplot(drift_data, aes(x = position, y = drift_score, color
    = first_genre)) +
438   # Add faint points to show the raw data variance
439   geom_point(alpha = 0.05, size = 0.5) +
440   # Add robust trend lines
441   geom_smooth(method = "lm", se = TRUE, size = 1.2) +
442   scale_color_brewer(palette = "Set1", name = "Seed Genre") +
443   facet_wrap(~ first_genre, labeller = label_both) +
444   labs(
445     title = "Drift Dynamics by Genre Cluster",
446     subtitle = "Interaction Analysis: Classical (5) shows negative
    slope (stickiness) vs. Pop (1) drift",
447     x = "Song Position",
448     y = "Euclidean Distance"
449   ) +
450   theme_minimal() +
451   theme(legend.position = "none") # Facets make legend redundant
452
453 print(p_genre)
454
455 #
    ================================================================================
456 # PHASE 1: SETUP & LIBRARIES
457 #
    ================================================================================
458 # Ensure all necessary packages are loaded
459 if(!require(tidyverse)) install.packages("tidyverse")
460 if(!require(ggfortify)) install.packages("ggfortify")
461 if(!require(broom)) install.packages("broom")
462 if(!require(knitr)) install.packages("knitr")
463
464 library(tidyverse)
465 library(ggfortify)
```

```r
library(broom)
library(knitr)

# 1. Load Data
# We use 'row.names = 1' to handle the index column correctly
df_raw <- final_dataset

# 2. Define Features
audio_vars <- c("danceability", "energy", "loudness", "speechiness",
                "accousticness", "instrumentalness", "liveness",
                "valence", "tempo")

# 3. Define a "Clean Theme" for Plots (Minimalist, Publication-Ready)
clean_theme <- theme_minimal() +
  theme(
    panel.grid.minor = element_blank(),       # Remove minor grid lines
    panel.grid.major = element_line(color = "grey92"), # Faint major lines
    plot.title = element_text(face = "bold", size = 14),
    axis.title = element_text(face = "bold", size = 11),
    legend.position = "bottom",               # Move legend to bottom
    strip.text = element_text(face = "bold")  # Bold headers
  )

# ================================================================================

# PHASE 2: DATA AGGREGATION (Matching Methodology Section 3.5)
# ================================================================================

# "Inference was conducted at the session level using aggregated scores..."
# We calculate the mean audio profile for each user, ignoring the seed songs (pos < 2)

df_session <- df_raw %>%
  filter(position >= 2) %>%
  group_by(user, first_genre, age, gender) %>%
  summarise(across(all_of(audio_vars), mean, na.rm = TRUE), .groups = "drop") %>%
  mutate(
    # Convert numerical codes to readable labels
    first_genre = factor(first_genre,
                         levels = c(1, 2, 3, 4, 5),
                         labels = c("Pop", "Urban", "Electronic", "Rock", "Classical")),
```

```
    age = as.factor(age),
    gender = as.factor(gender)
  )

# ============================================================================

# PHASE 3: PCA (Dimensionality Reduction)
# ============================================================================

# We run PCA on the aggregated session data to create orthogonal variables

# --- FIX: Added 'dplyr::' to prevent error ---
pca_res <- prcomp(df_session %>% dplyr::select(all_of(audio_vars)), scale. = TRUE)

# Extract the PC scores (The new variables for MANOVA)
df_pca_session <- bind_cols(
  df_session %>% dplyr::select(user, first_genre, age, gender),
  as.data.frame(pca_res$x[, 1:3]) # Keep PC1, PC2, PC3
)

# Print Variance Explained (To confirm > 80% assumption)
var_explained <- summary(pca_res)$importance[2, 1:3]
print("--- TABLE: Variance Explained ---")
print(var_explained)

# ============================================================================

# PHASE 4: MANOVA (Multivariate Analysis of Variance)
# ============================================================================

# Model: PC_Score ~ Genre + Age + Gender + Interactions

model_manova <- manova(cbind(PC1, PC2, PC3) ~ first_genre * age * gender, data = df_pca_session)

# Use Pillai's Trace (Robust to covariance violations, as per methodology)
manova_results <- summary(model_manova, test = "Pillai")

print("--- RESULTS: Session-Level MANOVA (Pillai's Trace) ---")
print(manova_results)
```

```
539
540 #
    ================================================================================
541 # PHASE 5: CLEAN PLOTS (Minimalist)
542 #
    ================================================================================
543
544 # PLOT A: The Feature Distribution (Shadow Plot)
545 # Visualizes the density of the aggregated data
546 plot_data_long <- df_session %>%
547   dplyr::select(first_genre, all_of(audio_vars)) %>%
548   pivot_longer(cols = all_of(audio_vars), names_to = "Feature", values_
      to = "Value")
549
550 p_dist <- ggplot(plot_data_long, aes(x = Value, fill = first_genre)) +
551   geom_density(alpha = 0.4, size = 0) + # Size 0 removes the outline
      noise
552   facet_wrap(~ Feature, scales = "free") +
553   scale_fill_brewer(palette = "Set1") +
554   labs(
555     title = "Figure 1: Audio Feature Distributions by Genre",
556     subtitle = "Session-level averages show distinct acoustic profiles"
      ,
557     x = NULL, y = "Density"
558   ) +
559   clean_theme +
560   theme(axis.text.y = element_blank()) # Remove Y-axis numbers for
      cleanliness
561
562 print(p_dist)
563
564 # PLOT B: The PCA Biplot (Structure of Recommendations)
565 # Visualizes the separation of genres in 2D space
566
567 # Create a simplified centroids dataframe for plotting ellipses nicely
568 centroids <- df_pca_session %>%
569   group_by(first_genre) %>%
570   summarise(PC1 = mean(PC1), PC2 = mean(PC2))
571
572 p_biplot <- ggplot(df_pca_session, aes(x = PC1, y = PC2, color = first_
      genre)) +
573   # 1. User Dots (Small and transparent to reduce noise)
574   geom_point(size = 1.5, alpha = 0.3) +
575
576   # 2. Confidence Ellipses (The "Filter Bubbles")
```

```
  stat_ellipse(level = 0.95, size = 1) +

  # 3. Label the Centroids (Instead of a legend, we label the blobs directly)
  geom_text(data = centroids, aes(label = first_genre),
            color = "black", fontface = "bold", size = 5, check_overlap = TRUE) +

  scale_color_brewer(palette = "Set1") +
  labs(
    title = "The Structure of Algorithmic Recommendations",
    subtitle = "PCA Projection of Session Centroids (PC1 vs PC2)",
    x = paste0("Intensity Dimension (", round(var_explained[1]*100, 1), "%)"),
    y = paste0("Rhythmic Dimension (", round(var_explained[2]*100, 1), "%)")
  ) +
  clean_theme +
  theme(legend.position = "none") # Remove legend entirely (labels are better)

print(p_biplot)

# ================================================================================

# PHASE 6: POST-HOC ANALYSIS (The "Missing Link")
# ================================================================================


# 1. LINEAR DISCRIMINANT ANALYSIS (LDA)
# This answers: "How perfectly can we separate the genres?"
library(MASS)
lda_model <- lda(first_genre ~ PC1 + PC2 + PC3, data = df_pca_session)

print("--- TABLE: Group Means on Canonical Variables ---")
print(lda_model$means)
# INTERPRETATION: Look at the values.
# If Classical has PC1 = -4 and Pop has PC1 = +2, that's your headline.

# 2. PCA LOADINGS (Unlocking the "Black Box")
# This answers: "What does PC1 actually mean?"
loadings <- pca_res$rotation[, 1:3]
print("--- TABLE: PCA Loadings (Correlations) ---")
print(round(loadings, 2))
# INTERPRETATION:
```

```
615 # If PC1 has Energy (+0.5) and Acousticness (-0.5), call it the "
        Intensity Axis".
616
617 # 3. PAIRWISE MANOVA (Optional but rigorous)
618 # If you really want to prove Pop != Urban
619 # We use a Bonferroni correction to be safe.
620 pairwise_res <- pairwise.t.test(df_pca_session$PC1, df_pca_session$
        first_genre,
621                                 p.adjust.method = "bonferroni")
622 print("--- TABLE: Pairwise Comparisons (PC1 Dimension) ---")
623 print(pairwise_res)
```

Listing 1: R Script for Statistical Analysis